



Elektrotehnički fakultet
Univerzitet u Banjoj Luci

IZVJEŠTAJ PROJEKTOG ZADATAKA

iz predmeta

SISTEMI ZA DIGITALNU OBRADU SIGNALA

Student:
Smiljanić Dejana, 1173/21

Mentori:
prof. dr Mladen Knežić
prof. dr Mitar Simić
ma Vedran Jovanović
dipl. inž. Dimitrije Obradović

Januar 2026. godine

Sadržaj

1. Opis projektnog zadatka	3
2. Izrada projektnog zadatka	4
2.1 Pretprocesiranje i učitavanje slike	4
2.2 Konverzija slike u <i>grayscale</i> i centriranje oko nule	5
2.3 DCT algoritam	6
2.4 Kvantizacija	8
2.5 Cik-cak transformacija	8
2.6 Kodovanje AC i DC koeficijenata pomoću RLE algoritma	8
2.7 Serijalizacija dobijenih rezultata i dekompresija	9
2.8 Upravljenje LE diodama	9
2.9 Optimizacija	10
3. Zaključak	12

1. Opis projektnog zadatka

U okviru projektnog zadatka pod nazivom „Kompresija slike korištenjem diskretne kosinusne transformacije na ADSP-21489 razvojnoj platformi” potrebno je realizovati sistem za kompresiju *grayscale* slika na razvojnom okruženju ADSP-21489 korištenjem diskretne kosinusne transformacije (u daljem tekstu DCT).

Kompresija se u osnovi zasniva na JPEG standardu, ali određeni koraci biće izostavljeni radi jednostavnosti. Modifikovana metoda kompresije koja se koristi u ovom slučaju svodi se na sljedeći algoritam:

- RGB sliku upisati na razvojnu platformu i izvršiti njenu konverziju u *grayscale*
- Centrirati piksele oko nule, segmentirati sliku, te primjeniti DCT i kvantizaciju. Matricu kvantizacije odabrati u skladu sa željenim stepenom kompresije. Takođe, ista matrica kvantizacije neophodna je da se izvrši dekompresija slike.
- Primjeniti cik-cak transformaciju te izvršiti kodovanje DC i AC koeficijenata.
- Serijalizovati dobijene rezultate u proizvoljnom formatu.
- Analizirati dobijene rezultate u *Python* ili *MATLAB* programskom jeziku korištenjem odgovarajućih metrika. Izvršiti dekompresiju slike u *MATLAB*-u ili *Python*-u, i prikazati MSE (*Mean Squared Error*) u odnosu na originalnu sliku.

Rad sistema potrebno je testirati na odgovarajućem skupu slika, pri čemu slike treba da budu odabrane tako da se jasno demonstrira rad korištenog algoritma. Centralni dio projektnog zadatka, osim implementacije traženih koraka, jeste optimizacija navedene metode kompresije. Prilikom optimizacije potrebno je detektovati *bottle neck* u kodu, to jest koji dio čitavog sistema zahtijeva najviše procesorskih ciklusa i njemu posvetiti posebnu pažnju tokom optimizacije. Takođe, potrebno je izvršiti mjerenje vremena izvršavanja značajnijih funkcija unutar projekta.

2. Izrada projektnog zadatka

Implementacija traženih funkcionalnosti odvijala se kroz nekoliko faza, od kojih svaka odgovara određenom dijelu sistema, a to su:

1. Pretprocesiranje ulazne slike i učitavanje iz BMP formata u niz RGB vrijednosti.
2. Konverzija iz RGB u *grayscale* učitane slike, te centriranje oko nule.
3. Segmentiranje slike na blokove, implementacija DCT algoritma i primjena na centriranu sliku.
4. Primjena kvantizacije na obrađenu sliku.
5. Primjena cik-cak transformacije.
6. Kodovanje DC i AC koeficijenata pomoću RLE algoritma.
7. Serijalizacija dobijenih rezultata u formi tekstualne datoteke.
8. Dekompresija slike kroz *Python* skriptu i vizuelizacija dobijenih rezultata.
9. Analiza dobijenih rezultata (računanje stepena uštede, analiza dobijene srednje kvadratne greške, vizuelna procjena originalne i dekompresovane slike).
10. Prikaz progressa obrade korištenjem LE dioda

Prva i posljednja faza ne izvršavaju se na razvojnoj platformi, ali su usko vezane za funkcionisanje sistema, tako da će biti obrađene samo u kontekstu njihovog značaja za ispravan rad sistema.

2.1. Pretprocesiranje i učitavanje slike

Ulazna slika izabrana je tako da bude u BMP formatu. Ovaj format izabran je iz razloga što slika predstavlja u vidu bitske mape R, G i B komponentata, sa fiksnom veličinom zaglavlja. Kako su slike nekompresovane, nije komplikovano dobiti sirove podatke o bojama koji su potrebni za učitavanje slike na razvojnu platformu, a nema ni gubitka informacija (takozvani *lossless* format). Slike u BMP formatu mogu se prikazati pomoću standardnih editora slika na Windows operativnom sistemu, što je dodatni motiv za korištenje.

Korišten je 24-bitni BMP format, a podaci o slikama su ekstrahovani kroz C program `bmp_loader.c`. Pri ekstrahovanju podataka vrši se provjera dimenzija slike, parsira BMP zaglavlje i učitavaju dobijeni podaci na razvojnu platformu u vidu `image_rgb.h` i `image_rgb.c` datoteke. Naravno, potrebno je kompajlirati projekat sa ovim (ažuriranim) datotekama i potom učitati izvršni fajl na razvojnu ploču.

2.2. Konverzija slike u *grayscale* i centriranje oko nule

RGB slika učitana je na razvojnu ploču u vidu niza `uint8_t` podataka. Navedeni tip podataka korišten je iz razloga što je poznato da RGB komponente mogu uzimati vrijednosti od 0 do 255. Konverzija u *grayscale* zasniva se na ljudskoj percepciji boja. Receptori u oku su najosjetljiviji na zelenu boju, pa pri konverziji treba voditi računa da se naglašavaju boje na koje je oko osjetljivije kako bi slika izgledala prirodno i nakon konverzije u crno-bijeli spektar. Formula za konverziju standardno se zapisuje kao

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \quad (2.1)$$

Konverzija je izvršena u `grayscale.c`, pri čemu je korišten *fixed-point* format podataka. Kada se (2.1) zapiše u *Q0.8 fixed-point* formatu, ona ima sljedeći oblik:

$$Y = 77 \cdot R + 150 \cdot G + 29 \cdot B \quad (2.2)$$

Funkcija kojom se implementira opisano data je ispod.

Listing 1: ConvertRgbToGrayscale iz `grayscale_converter.c`

```

1 void ConvertRgbToGrayscale(void) {
2     /* ... */
3     for (uint32_t i = 0; i < total_pixels; i++) {
4         uint8_t r = rgb_image_buffer[i * 3 + 0];
5         uint8_t g = rgb_image_buffer[i * 3 + 1];
6         uint8_t b = rgb_image_buffer[i * 3 + 2];
7
8         int32_t gray = ((FIXED_R * r + FIXED_G * g + FIXED_B *
9             b) >> 8);
10
11         grayscale_image_buffer_g[i] = (int8_t)(gray - 128);
12     }
13 }
```

Centriranje oko nule predstavlja ništa drugo nego oduzimanje 128 od izračunate *grayscale* vrijednosti (linija 12 u priloženom kodu).

Bolje performanse mogle bi se postići kada bi R, G i B komponente imale drugačiji raspored u ulaznim podacima, npr. sve R komponente, potom sve G, potom B. Tada bi čitanje moglo da se radi paralelno, odnosno, moglo bi se iskoristiti SIMD izvršavanje.

2.3. DCT algoritam

Slika se obrađuje tako što se podijeli na blokove veličine 8x8. Svaki blok se transformiše nezavisno. Rezultat DCT transformacije je matrica koeficijenata, gdje:

- gornji lijevi element predstavlja DC komponentu (prosječnu vrijednost bloka),
- ostali elementi predstavljaju AC komponente koje opisuju promjene intenziteta u različitim frekvencijama.

Matematički, dvodimenzionalna DCT za blok veličine $N \times N$ definisana je izrazom:

$$F(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \cos \left[\frac{(2y+1)v\pi}{2N} \right] \quad (2.3)$$

gdje je $f(x, y)$ ulazna vrijednost piksela, $F(u, v)$ DCT koeficijent, dok su $\alpha(u)$ i $\alpha(v)$ faktori normalizacije.

Faktori normalizacije $\alpha(k)$, prikazani u jednačini (2.4), osiguravaju da transformacija bude ortonormalna:

$$\alpha(k) = \begin{cases} \frac{1}{\sqrt{N}}, & k = 0 \\ \sqrt{\frac{2}{N}}, & k \geq 1 \end{cases} \quad (2.4)$$

U matričnom obliku, 2D DCT se tada može efikasno izračunati kao $F = C f C^T$. Pri implementaciji je korištena ova formula, a potrebne konstante su unaprijed izračunate kroz C program na PC-u i sačuvane u vidu tabela smještenih u programsku memoriju (u daljem tekstu PM).

Umjesto direktnog računanja dvodimenzionalne transformacije, DCT se sprovodi u dva uzastopna koraka: prvo se vrši transformacija po jednom pravcu, a zatim po drugom. Iako postoje kompleksniji algoritmi sa manjim brojem operacija, kao što je AAN (*Arai, Agui, Nakajima*), izabrani pristup omogućava maksimalnu iskoristivost SIMD jedinica SHARC procesora kroz paralelnu obradu elemenata matrice. Upravo mogućnost SIMD izvršavanja predstavlja jednu od tehnika optimizacija koja je korištena pri optimizaciji izvršavanja DCT algoritma koji prema broju ciklusa predstavlja jedan od dijelova sistema koji troši najviše procesorskog vremena (uz *grayscale* konverziju).

Listing 2: DctBlockProcess iz dct.c

```
1 void DctBlockProcess(const int8_t* input_p, float* output_p,
2                      uint16_t start_x, uint16_t start_y)
3 {
4     /* ... */
5     /* Load 8x8 image block and convert to float */
6     for (r = 0; r < 8; r++)
7     {
8         const int8_t* src = &input_p[(start_y + r) * IMG_WIDTH +
9                                     start_x];
10        float* dst = &block[r * 8];
11        dst[0]=(float)src[0]; dst[1]=(float)src[1]; /* ... */
12    }
13    /* First multiplication: T * block */
14    for (r = 0; r < 8; r++) {
15        for (c = 0; c < 8; c++) {
16            float sum;
17            const float* row_T = &T[r][0];
18            const float* col_B = &block[c];
19
20            sum = row_T[0] * col_B[0];
21            sum += row_T[1] * col_B[8]; /* ... */
22
23            s_inter[r * 8 + c] = sum;
24        }
25    }
26    /* Second multiplication: intermediate * T^T */
27    for (r = 0; r < 8; r++) {
28        for (c = 0; c < 8; c++) {
29            float sum;
30            const float* row_I = &s_inter[r * 8];
31            const float* row_T = &T[c][0];
32
33            sum = row_I[0] * row_T[0];
34            sum += row_I[1] * row_T[1]; /* ... */
35
36            output_p[r * 8 + c] = sum;
37        }
38    }
```

2.4. Kvantizacija

Kvantovanje se vrši koeficijent po koeficijent pomoću matrice kvantovanja $Q(u, v)$. Kvantovani koeficijenti $\hat{F}(u, v)$ dobijaju se relacijom:

$$\hat{F}(u, v) = \text{round} \left(\frac{F(u, v)}{Q(u, v)} \right) \quad (2.5)$$

Nakon kvantovanja, DCT koeficijenti u bloku $N \times N$ raspoređeni su u matrici gdje se NF komponente nalaze u gornjem lijevom uglu, a VF prema donjem desnom uglu. NF koeficijenti najčešće zadržavaju značajne (nenulte) vrijednosti, dok se velik dio VF koeficijenata nakon kvantovanja pretvara u nule.

Pri implementaciji kvantizacije nije korištena operacija dijeljenja, već operacija množenja sa recipročnom vrijednošću pošto je na korištenoj platformi operacija množenja mnogo brža od operacije dijeljenja. Recipročne vrijednosti čuvaju se u tabeli Q_INV u PM, tako da se vrijednosti samo čitaju iz memorije. Takođe, korišteno je zaokruživanje na najbliži cijeli broj koje je implementirano uz pomoć osobine kastovanja `float` podatka u `integer` podataka i dodavanjem odnosno oduzimanjem faktora 0.5 u zavisnosti od znaka podatka. Tako npr. broj 3.6 postaje 4.1, pa se zaokružuje na 4, dok broj -3.6 postaje -4.1, pa se zaokružuje na -4.

2.5. Cik-cak transformacija

Cik-cak (*zig-zag*) transformacija predstavlja unaprijed definisano čitanje koeficijenata iz matrice u niz tako da se koeficijenti poredaju od nižih ka višim frekvencijama. Time se postiže da se većina nenulih vrijednosti nađe na početku niza, dok se u drugom dijelu niza tipično pojavljuju dugi nizovi nula. Ovakav raspored je povoljan za kasnije kodovanje, naročito za *run-length* koje se ovdje koristi. Matrica cik-cak indeksa takođe se nalazi u PM, a njene vrijednosti uzete su tako da odgovaraju preporukama iz korištene literature.

2.6. Kodovanje AC i DC koeficijenata pomoću RLE algoritma

Posljednja faza kompresije na razvojnoj platformi je RLE kodovanje, realizovano u `rl_encoder.c`.

S obzirom na veliku korelaciju između susjednih blokova, DC koeficijent se koduje diferencijalno. U memoriji se čuva samo razlika u odnosu na prethodni blok. AC koeficijenti, prethodno raspoređeni cik-cak transformacijom, koduju se upotrebom parova (*zero_num*, *coeff_value*). Svaki par predstavlja broj uzastopnih nula (*zero_num*) koje prethode nenultom koeficijentu (*coeff_value*).

Listing 3: Implementacija RLE (glavna petlja)

```

1  for(i = 1; i < 64; i++) {
2      if(zigzag_p[i] == 0) {
3          zero_num_i++;
4      } else {
5          RLE_output_p->AC_coeffs[ac_i].zero_num = zero_num_i;
6          RLE_output_p->AC_coeffs[ac_i].coeff_value = zigzag_p[i];
7          ac_i++;
8          zero_num_i = 0;
9      }
10 }

```

Proces se završava umetanjem *End-Of-Block* (EOB) *flega*, koji je indikator da su svi preostali koeficijenti u bloku jednaki nuli.

2.7. Serijalizacija dobijenih rezultata i dekompresija

Nakon što se izvrši kompresija svih blokova slike na razvojnoj platformi, neophodno je rezultate prenijeti na PC radi provjere i dekompresije. Ovaj proces je implementiran kroz funkciju `DumpRleToFile`.

Serijalizacija se vrši generisanjem tekstualne datoteke *custom* formata koji omogućava rekonstrukciju slike kroz jednostavno parsiranje pomoću *Python* skripte. Format zapisa za svaki blok obuhvata redni broj bloka, diferencijalno kodovanu DC vrijednost, niz AC parova (*zero_num*, *coeff_value*) i završni EOB fleg.

Na strani PC vrši se dekompresija pomoću *Python* skripte, te se vrši prikaz originalne slike u RGB, te u *grayscale* formatu, prikaz dekompresovane slike i srednje kvadratne greške. Na osnovu ovih podataka moguće je izvršiti provjeru i ocijeniti kvalitet kompresovane slike.

2.8. Upravljenje LE diodama

U ovom modulu implementirana je kontrola onboard LE dioda korišćenjem SRU jedinice i FLAG/GPIO mehanizama. LED1–LED3 kontrolišu se preko FLAG izlaza procesora, dok su LED4–LED8 direktno povezane putem DAI portova. Funkcija `LedInit` vrši inicijalizaciju sistema, konfiguraciju SRU rutiranja i postavlja LED u početno isključeno stanje. Funkcija `LedOn` omogućava uključivanje pojedinačne LE diode na osnovu njenog indeksa, dok `LedsOff` služi za gašenje svih DAI-kontrolisanih LE dioda.

2.9. Optimizacija

Svaka od pobrojanih faza, osim serijalizacije, analizirana je sa ciljem detektovanja *bottle-neck*-a, odnosno dijela sistema koji troši najviše procesorskih ciklusa. Svaka faza optimizovana je u skladu sa mogućnostima primjene tehnika optimizacija i na osnovu rezultata testiranja izvršavanja optimizovanih dijelova koda, a naročita pažnja posvećivala se optimizaciji konverzije iz RGB u *grayscale* i DCT algoritmu koji predstavljaju *bottle-neck*.

Kako bismo mogli uvidjeti efekat tehnika optimizacija, potrebno je da izbrojimo koliko procesorskih ciklusa troši kod bez optimizacije. Mjerenje ciklusa može se izvršiti pomoću preprocesorske direktive `DO_CYCLE_COUNTS` i zaglavlja `cycle_count.h`.

Listing 4: Mjerenje procesorskih ciklusa (neoptimizovana verzija)

```
Cycle num for grayscale conversion: 9175073
Cycle num for processing: 18463864
Cycle num for serialization: 9333840
```

Sada možemo omogućiti optimizaciju koda kroz *CrossCore Embedded Studio* i iskoristiti dostupne pragma direktive, te reorganizovati kod gdje je to moguće.

Pošto se za petlju kojom se izvršava konverzija u *grayscale* ne može iskoristiti SIMD izvršavanje, dodaćemo `#pragma loop_count (64, 65535, 8)` kako bismo kompajleru dali informacije o minimalnom i maksimalnom broju iteracija, kao i o tome da je broj iteracija uvijek djeljiv sa 8, redom.

Listing 5: Optimizacija ConvertRgbToGrayscale

```
1 #pragma loop_count (64, 65535, 8)
2     for (uint32_t i = 0; i < total_pixels; i++)
3     { /*...*/ }
```

Navedenu pragma direktivu možemo dodati i u kodu kojim se implementira DCT algoritam. Takođe, u ovom slučaju možemo iskoristiti i SIMD izvršavanje koje možemo sugerisati kompajleru pomoću `#pragma SIMD_for` pragma direktive. Indirektna tehnika optimizacije predstavlja i sam način implementacije algoritma (umjesto višestrukih poziva kosinusne funkcije vrijednosti su unaprijed izračunate i samo se čitaju).

Listing 6: Optimizacija DctBlockProcess

```
1 #pragma SIMD_for
2 #pragma loop_count (8, 8, 8)
3 for (c = 0; c < 8; c++) { /* ... */ }
```

Kako je tekstom projektnog zadatka traženo da se svaka faza implementacije optimizuje koliko je to moguće, priložićemo i tehnike optimizacije korištene u ostalim fazama.

Implementacija RLE algoritma sadrži uslovno grananje, što se smatra „ubicom” performansi. Kako nije moguće izbjeći navedeno u ovom slučaju, možemo kompajleru sugerisati da postoji

veća vjerojatnoća određenog ishoda (`true` ili `false`) uslova grananja. U tu svrhu definiše se `choices.h` sa odgovarajućim makroima.

Listing 7: Definicija EXPRA makroa u `choices.h`

```
1 #if EXPRS == 1
2     #define EXPRA(_x) expected_true(_x)
3 /* ... */
4 #endif
```

Korištenje ovih makroa opravdano je jer se pri kvantizaciji većina AC koeficijenata pretvori u 0, pa je brojanje nula efikasnije nakon upotrebe ovih makroa.

Listing 8: Primjena `expected_true` funkcije u RLE algoritmu

```
1 if (EXPRA(zigzag_p[i] == 0)) {
2     zero_num_i++;
3 }
```

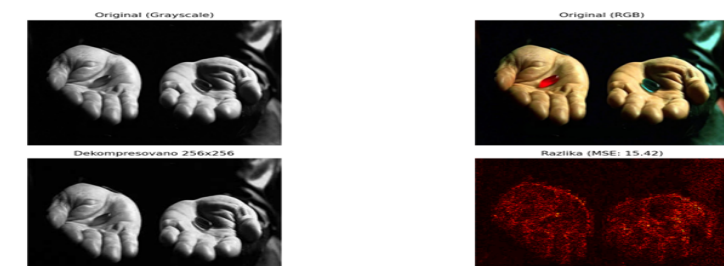
U većini petlji od kojih se sastoji sam program može se iskoristiti pragma direktiva `#pragma SIMD_for` naročito ako fiksiramo dimenzije slike na 256x256 (što je ovdje i učinjeno) i unaprijed predvidimo broj iteracija uzevši to u obzir. Kada se izmjeri broj procesorskih ciklusa potreban za izvršavanje koda sa ovim izmjenama, dobijaju se rezultati prikazani u Tabeli 1.

Tabela 2.1 – Mjerenje procesorskih ciklusa po modulima (optimizovana verzija)

Modul	Broj Procesorskih Ciklusa
Grayscale konverzija	3 932 201
Kompresija slike i kodovanje	4 447 424

Vidimo da se broj procesorskih ciklusa smanjio tri puta kada je riječ o *grayscale* konverziji i četiri puta kada je riječ o DCT algoritmu.

Nakon obrade slike, pomoću pomenute *Python* skripte izvršeno je parsiranje podataka i vizuelizacija rezultata. Step kompresije slike koja je uzeta za primjer iznosi 8.42 : 1, što je procentualno 88.12%. Vizuelnom procjenom slike ne može se vidjeti drastičan pad kvalitete slike, a srednje kvadratna greška je relativno niske vrijednosti za dimenziju slike 256x256.



Slika 2.1 – Prikaz originalne (RGB i grayscale), dekompresovane slike i MSE

3. Zaključak

U okviru ovog projektnog zadatka uspješno je realizovan sistem za kompresiju *grayscale* slike zasnovan na diskretnoj kosinusnoj transformaciji (DCT) na razvojnoj platformi ADSP-21489. Implementirani algoritam prati osnovne principe JPEG standarda, uz određena pojednostavljenja.

Poseban akcenat u radu stavljen je na optimizaciju izvršavanja algoritma na DSP arhitekturi. Analizom potrošnje procesorskih ciklusa detektovani su dijelovi sistema koji predstavljaju *bottle-neck*, prvenstveno konverzija u *grayscale* i DCT algoritam. Primjenom dostupnih tehnika optimizacije, kao što su upotreba `#pragma loop_count`, `#pragma SIMD_for`, reorganizacija koda i korištenje unaprijed izračunatih tabela u programskoj memoriji, postignuto je značajno smanjenje broja procesorskih ciklusa.

Naredni koraci ka poboljšanom sistemu uključivali bi efikasnije učitavanje RGB slika (npr. pomoću UART interfejsa), korišćenje DMA za pribavljanje podataka iz memorije, efikasniju serijalizaciju (UART, SPI interfejs), entropijsko kodovanje i naprednije DCT algoritme.

Literatura

- [1] Analog Devices, “CCES 2.9.0 C/C++ Compiler Manual for SHARC Processors,” 2019. [Online]. Available: <https://www.analog.com/media/en/dsp-documentation/software-manuals/cces-sharccompiler-manual.pdf>
- [2] Analog Devices, “ADSP-21489 EZ-Board™ Evaluation System Manual,” 2010. [Online]. Available: https://www.analog.com/media/en/dsp-documentation/evaluation-kit-manuals/ADSP_21489_EZ_Board_Manual_Rev_1_0_April_2010.pdf
- [3] Zhang Jiaqi, “Understanding DCT and Quantization in JPEG compression,” 2023. [Online]. Available: <https://dev.to/marycheung021213/understanding-dct-and-quantization-in-jpeg-compression-1col>
- [4] Nathan Liesch, “THE BMP FILE FORMAT,” [Online]. Available: http://www.ece.ualberta.ca/~elliott/ee552/studentAppNotes/2003_w/misc/bmp_file_format/bmp_file_format.htm